



**HOCHSCHULE
HANNOVER**
UNIVERSITY OF
APPLIED SCIENCES
AND ARTS

SafeDigi

Zeitplanung

Projekt: darkIT Spielesammlung

Gruppe: 03

Entwickler: Heysem Omar

Semester: 03. Semester BIN

Betreuer: Andreas Holitschke

31. Oktober 2025

Inhaltsverzeichnis

Zeitplanung	- 3 -
Inhaltsverzeichnis.....	- 3 -
1 Projektübersicht.....	- 3 -
1.1 Projektziele	- 3 -
1.2 Projektumfang	- 3 -
1.3 Technologie-Stack.....	- 3 -
1.4 Zeitrahmen	- 3 -
2 Schritte zur Erstellung des Zeitplans.....	- 4 -
2.1 1. Projektziele definieren	- 4 -
2.2 2. Aufgaben identifizieren.....	- 4 -
2.3 3. Aufgaben priorisieren	- 4 -
2.4 4. Ressourcen zuweisen.....	- 4 -
2.5 5. Zeitaufwand schätzen	- 4 -
2.6 6. Meilensteine festlegen.....	- 4 -
3 Detaillierter Zeitplan	- 4 -
4 Benötigte Ressourcen	- 6 -
4.1 Software-Ressourcen.....	- 6 -
4.2 Fachliche Ressourcen.....	- 6 -
4.3 Materialien.....	- 6 -
5 Risikomanagement.....	- 7 -
6 Meilensteine und Erfolgskriterien	- 7 -
6.1 Haupt-Meilensteine	- 7 -
6.2 Erfolgskriterien.....	- 7 -
7 Überwachung und Berichtsmethoden	- 7 -

7.1 Fortschrittskontrolle.....	- 7 -
7.2 Anpassungen.....	- 7 -
 8 Fazit.....	 - 8 -

Zeitplanung

Programmierprojekt (DarkIT) **SafeDigi** Ein interaktives Lernspiel zur Förderung der digitalen Sicherheit

Inhaltsverzeichnis

1 Projektübersicht

1.1 Projektziele

Das Projekt **SafeDigi** ist ein Java-basiertes Lernspiel, das Benutzer dabei unterstützt, grundlegende Konzepte der digitalen Sicherheit spielerisch zu erlernen und das Bewusstsein für Cyber-Sicherheit zu stärken.

1.2 Projektumfang

- **Gameplay:** Spieler beantworten Fragen zu digitaler Sicherheit in verschiedenen Schwierigkeitsgraden.
- **Level-System:** 3+ Level mit steigender Komplexität, verwaltet durch separate Generatoren (LevelXQuestionGenerator.java).
- **Spiellogik:** Steuerung des Spielablaufs, der Punktezahl und des Level-Fortschritts (SafeDigiGameLogic.java).
- **Charakter-Panel:** Visuelle Darstellung des Charakters oder des Spielstatus (CharacterPanel.java).
- **Audio:** Hintergrundmusik und Sound-Effekte (SoundPlayer.java).
- **UI/UX:** Moderne, benutzerfreundliche Oberfläche mit klarem Fokus auf die Fragen.

1.3 Technologie-Stack

- **Programmiersprache:** Java
- **GUI-Framework:** Java Swing (impliziert durch die Dateinamen)
- **Versionskontrolle:** Git/GitHub
- **IDE:** IntelliJ IDEA oder Eclipse

1.4 Zeitrahmen

12 Wochen (ca. 3 Monate)

2 Schritte zur Erstellung des Zeitplans

2.1 1. Projektziele definieren

Klare Ziele und Anforderungen des Projekts wurden festgelegt: Ein interaktives Lernspiel zur digitalen Sicherheit mit einem mehrstufigen Frage-Antwort-System.

2.2 2. Aufgaben identifizieren

Alle notwendigen Aufgaben wurden in folgende Phasen unterteilt: * Projektinitiierung und Planung * Anforderungsanalyse und Design-Review * Implementierung (Refactoring, Features) * Testing und Optimierung * Dokumentation * Projektabschluss

2.3 3. Aufgaben priorisieren

Die Aufgaben wurden nach Abhängigkeiten geordnet: 1. Design-Review vor Implementierung neuer Features. 2. Kernfunktionalität (Logik) vor Features (Sound, UI-Polish). 3. Testing parallel zur Entwicklung. 4. Dokumentation kontinuierlich.

2.4 4. Ressourcen zuweisen

Entwickler: Heysem Omar (Einzelprojekt) Ressourcen: IDE, Java SDK, Git, UML-Tools, Audio-Dateien

2.5 5. Zeitaufwand schätzen

Jede Phase wurde auf ca. 1 Woche geschätzt, mit Pufferzeit für unvorhergesehene Probleme.

2.6 6. Meilensteine festlegen

Wichtige Meilensteine wurden definiert (siehe Tabelle im nächsten Abschnitt).

3 Detaillierter Zeitplan

Woche	Aktivität	Aufgaben	Verantwortlich	Meilenstein
1	Projektinitiierung & Analyse	• Kick-off Meeting und Zieldefinition (SafeDigi Game) • Technologie-Stack bestätigen (Java/Swing)	Heysem Omar	Code-Basis analysiert und lauffähig (JAR-Datei)
2	Anforderungsanalyse & Design I	• Erstellung des Use Case Diagramm Prototypen	Heysem Omar	Use Case und Zustandsdiagramm

		• Erstellung des Zustandsdiagramms des Spiels • Detaillierte Analyse der Spielmechanik		fertiggestellt
3	Design II & Modellierung	• Erstellung des Sequenzdiagramms • Erstellung des Klassendiagramms • Finalisierung des Architektur-Entwurfs	Heysem Omar	Sequenz- und Klassendiagramm fertiggestellt
4	Planungsabschluss	• Finalisierung der Zeitplanung • Abgabe des Zeitplans und der Design-Dokumente	Heysem Omar	Zeitplanung und Design-Dokumente abgegeben
5	Implementierung Phase 1: Grundgerüst	• Implementierung des Grundgerüsts (JFrame, Panels) • Welcome Screen und Basis-Navigation • Vorbereitung der GameFacade und SafeDigiLogic Klassen	Heysem Omar	Grundgerüst steht
6	Implementierung Phase 2: Kern-Logik	• Implementierung der SafeDigiGameLogic (Punktezählung, Spielregeln) • Anbindung der QuestionGenerator an die Logik • Implementierung des Antwort-Feedbacks	Heysem Omar	Kern-Gameplay (Frage/Antwort-Logik) funktioniert
7	Implementierung Phase 3: Level & UI	• Implementierung der Level-Fortschrittslogik • Finalisierung des CharacterPanel.java (visuelle Darstellung) • Implementierung des GamePanels mit Fragen-Anzeige	Heysem Omar	Level-Fortschritt und Haupt-UI funktionsfähig
8	Implementierung Phase 4: Features	• Implementierung des Highscore-Systems (Speichern/Laden) • Implementierung eines Sterne- oder Bewertungssystems • Implementierung des SoundPlayer.java für Musik und Effekte	Heysem Omar	Alle Kern-Features sind implementiert

9	UI/UX Polish & Assets	• Visuelle Verbesserung der Benutzeroberfläche • Einbindung finaler Grafiken und Icons • Optimierung der Animationen und Übergänge	Heysem Omar	UI/UX ist finalisiert
10	Testing & Bug-Fixing Phase I	• Systematisches Testen aller Level und Fragen • Unit-Tests für Kernlogik • Behebung aller gefundenen Bugs und Logikfehler	Heysem Omar	Kernlogik ist fehlerfrei
11	Testing & Bug-Fixing Phase II & Dokumentation	• Integrationstests der gesamten Anwendung • Erstellung der Javadoc-Kommentare und README.md • Finalisierung der UML-Diagramme	Heysem Omar	Dokumentation erstellt und System stabil
12	Projektabschluss	• Erstellung der finalen, ausführbaren JAR-Datei • Durchführung des finalen Akzeptanztests • Abgabe vorbereiten (Bericht, Präsentation)	Heysem Omar	Projekt abgegeben

4 Benötigte Ressourcen

4.1 Software-Ressourcen

- **Entwicklungsumgebung:** IntelliJ IDEA oder Eclipse
- **Programmiersprache und Runtime:** Java SDK
- **Versionskontrolle:** Git/GitHub
- **Dokumentation:** PlantUML / Draw.io (für UML-Diagramme)

4.2 Fachliche Ressourcen

- Kenntnisse in Java und Java Swing
- Verständnis von OOP-Prinzipien und Entwurfsmustern
- Fachwissen im Bereich digitale Sicherheit/Lerninhalte

4.3 Materialien

- Sammlung von Fragen zur digitalen Sicherheit
- Audio-Dateien für Musik und Sound-Effekte
- Icons und Grafiken für UI-Elemente

5 Risikomanagement

Risiko	Wahrscheinlichkeit	Gegenmaßnahme
Logikfehler in SafeDigiGameLogic	Mittel	Frühzeitiges, umfassendes Unit-Testing der Logik-Klassen
Integrationsprobleme (z.B. SoundPlayer in SafeDigi)	Mittel	Modulare Entwicklung und regelmäßige Integrationstests
Zeitmangel für UI-Polish	Hoch	MVP-Ansatz: Kern-Gameplay vor UI-Features abschließen; Pufferzeit einplanen
Fragen-Qualität unzureichend	Mittel	Review der Fragen durch Fachexperten (oder Kommilitonen)

6 Meilensteine und Erfolgskriterien

6.1 Haupt-Meilensteine

1. **Woche 1:** Code-Basis importiert und verstanden.
2. **Woche 4:** Klassenstruktur refactored und Level-Design abgeschlossen.
3. **Woche 8:** Alle Kern-Features (Gameplay, Sound, Highscore) implementiert.
4. **Woche 10:** System getestet und stabil.
5. **Woche 12:** Projekt volldokumentiert und abgegeben.

6.2 Erfolgskriterien

Das Projekt gilt als erfolgreich, wenn: * Das Spiel mindestens 3 funktionierende Level mit dynamischen Fragen enthält. * Die Spiellogik (Punktezählung, Level-Fortschritt) fehlerfrei funktioniert. * Die Anwendung als ausführbare JAR-Datei lauffähig ist. * Eine vollständige Dokumentation (Javadoc, README) vorliegt. * Das Projekt termingerecht abgeschlossen wird.

7 Überwachung und Berichtsmethoden

7.1 Fortschrittskontrolle

- **Wöchentliche Selbstreflexion:** Überprüfung des Fortschritts anhand der Meilensteine.
- **Git-Commits:** Regelmäßige Commits mit aussagekräftigen Commit-Messages.
- **Task-Board:** Verwendung von GitHub Issues oder Trello zur Aufgabenverfolgung.

7.2 Anpassungen

Der Zeitplan kann bei Bedarf angepasst werden, wenn: * Unvorhergesehene technische Probleme auftreten. * Sich Anforderungen ändern. * Verzögerungen durch externe

Faktoren entstehen. Bei größeren Abweichungen werden Features priorisiert (MVP-Ansatz: Minimum Viable Product zuerst fertigstellen).

8 Fazit

Dieser Zeitplan dient als Leitfaden für die strukturierte Entwicklung und Finalisierung des **SafeDigi Game** über einen Zeitraum von 12 Wochen. Durch die klare Definition von Aufgaben, Meilensteinen und Risiken wird sichergestellt, dass das Projekt termingerecht und mit hoher Qualität abgeschlossen werden kann.

Projektziel: Ein funktionsfähiges, gut dokumentiertes Lernspiel zur digitalen Sicherheit, das als ausführbare JAR-Datei läuft und termingerecht abgegeben wird.

Der vorliegende Zeitplan bildet die Grundlage für ein erfolgreiches Projektmanagement und stellt sicher, dass alle wichtigen Aspekte der Softwareentwicklung – von der Anforderungsanalyse über die Implementierung bis hin zur Dokumentation – systematisch und nachvollziehbar durchgeführt werden.